

PROMPT TÉCNICO PARA ANTIGRAVITY - MÓDULO 5: API, WEBHOOKS E INTEGRACIONES

markdown

CONTEXTO DEL MÓDULO 5

Estás construyendo la **CAPA DE INTEGRACIÓN Y API PÚBLICA** de la plataforma SaaS de reservas.

Este módulo permite que sistemas externos (POS, Google Maps, Zapier) se conecten con nuestra plataforma.

Incluye una API RESTful segura, un sistema de Webhooks robusto, documentación interactiva y conectores nativos.

STACK TECNOLÓGICO:

- Backend: Next.js API Routes / Edge Functions
- Documentación: NextSwagger o tRPC / OpenAPI 3.0
- Cache & Rate Limiting: Upstash Redis
- Base de Datos: PostgreSQL (Prisma ORM)
- Webhooks: Svix (opcional) o lógica custom con intentos y firmas (HMAC)
- SDK: Generación automática para JS/Python
- Integraciones: Google Reserve API, Zapier CLI

ARQUITECTURA DE SEGURIDAD

REQUISITOS FUNCIONALES:

1. **Autenticación de API:**

- Cada restaurante tiene un ``API_KEY`` (Prefijo: ``res_live_...`` o ``res_test_...``).
- Autenticación vía Header: ``Authorization: Bearer <API_KEY>``.
- Opción de generar múltiples claves con permisos específicos (solo lectura, solo escritura).

2. **Rate Limiting (Límites de Uso):**

- Proteger contra abusos (DDoS, scraping).
- Límites por plan:
 - * Free: 100 requests/hora
 - * Pro: 1,000 requests/hora
 - * Enterprise: Ilimitado

- Headers de respuesta: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`.

3. **Validación de Firmas (Webhooks):**

- Firmas HMAC-SHA256 para asegurar que el webhook viene de nuestra plataforma.

- Header: `X-Webhook-Signature`.

REQUISITOS TÉCNICOS:

Middleware de Autenticación:

```
`typescript
// middleware/api-auth.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { prisma } from '@lib/prisma';
import { ratelimit } from '@lib/redis'; // Upstash
configuration

export async function middleware(request: NextRequest) {
  // Excluir rutas públicas y documentación
  if (request.nextUrl.pathname.startsWith('/api/docs') ||
    request.nextUrl.pathname.startsWith('/public')) {
    return NextResponse.next();
  }

  const apiKey =
request.headers.get('authorization')?.replace('Bearer ', '');

  if (!apiKey) {
    return NextResponse.json({ error: 'API Key requerida' }, {
status: 401 });
  }

  // Validar API Key en DB
  const keyRecord = await prisma.apiKey.findUnique({
    where: { key: apiKey },
    include: { restaurant: true }
  });

  if (!keyRecord || !keyRecord.isActive) {
    return NextResponse.json({ error: 'API Key inválida' }, {
status: 403 });
  }
}
```

```

    }

    // Rate Limiting
    const ip = request.ip || '127.0.0.1';
    const { success, limit, reset, remaining } = await
    ratelimit.limit(ip);

    if (!success) {
        return NextResponse.json({ error: 'Límite de requests
    excedido' }, { status: 429 });
    }

    // Pasar datos al request
    request.headers.set('x-restaurant-id',
    keyRecord.restaurantId);

    const response = NextResponse.next();
    response.headers.set('X-RateLimit-Limit', limit.toString());
    response.headers.set('X-RateLimit-Remaining',
    remaining.toString());
    response.headers.set('X-RateLimit-Reset', reset.toString());

    return response;
}

export const config = {
    matcher: '/api/v1/:path*',
};

```

SECCIÓN 1: API PÚBLICA (REST)

REQUISITOS FUNCIONALES:

Exponer endpoints para que terceros manipulen datos del restaurante.

Endpoints Principales:

1. Reservas (Bookings):

- GET /api/v1/bookings: Listar reservas (filtros: fecha, estado).
- POST /api/v1/bookings: Crear reserva desde un sistema externo (ej. llamada entrante al POS).
- PUT /api/v1/bookings/:id: Actualizar estado (Check-in, No-show, Cancelado).
- DELETE /api/v1/bookings/:id: Cancelar reserva.

2. Disponibilidad (Availability):

- GET /api/v1/availability: Consultar slots libres (para mostrar en app del restaurante o Google Maps).

3. Mesas (Tables):

- GET /api/v1/tables: Listar configuración de mesas.
- PUT /api/v1/tables/:id: Cambiar estado de una mesa (Ocupada/Libre).

4. Clientes (Customers):

- GET /api/v1/customers: Buscar clientes (por teléfono/email).
- POST /api/v1/customers: Registrar nuevo cliente (CRM Sync).

Formato de Respuesta Estándar:

```
json
{
  "success": true,
  "data": { ... },
  "meta": {
    "page": 1,
    "total": 100
  }
}
```

REQUISITOS TÉCNICOS:

Implementación con Zod y Next.js:

```
typescript
// app/api/v1/bookings/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { z } from 'zod';
import { createBookingSchema } from '@lib/validations';
import { createBookingService } from '@services/booking';

export async function POST(req: NextRequest) {
  try {
    const body = await req.json();

    // Validar payload
    const validatedData = createBookingSchema.parse(body);
    const restaurantId = req.headers.get('x-restaurant-id');

    // Lógica de negocio
    const booking = await createBookingService({
      ...validatedData,
      restaurantId,
      source: 'api', // Marca que vino por API
    });
  } catch (error) {
    // Manejar errores de validación o negocio
  }
}
```

```

    });

    return NextResponse.json({ success: true, data: booking },
    { status: 201 });
  } catch (error) {
    if (error instanceof z.ZodError) {
      return NextResponse.json({ error: 'Datos inválidos',
      details: error.errors }, { status: 400 });
    }
    return NextResponse.json({ error: 'Error interno' }, {
    status: 500 });
  }
}

```

SECCIÓN 2: SISTEMA DE WEBHOOKS

REQUISITOS FUNCIONALES:

Sistema para notificar a sistemas externos cuando ocurren eventos importantes.

Eventos Disponibles:

1. booking.created: Nueva reserva confirmada.
2. booking.updated: Se modificó una reserva.
3. booking.cancelled: Reserva cancelada.
4. booking.no_show: Cliente no asistió.
5. booking.checkin: Cliente llegó.
6. table.status_changed: Mesa cambió de estado.

Panel de Configuración de Webhooks (UI):

- Campo URL: `https://mi-restaurante.com/webhook`
- Checkboxes para seleccionar eventos (ej: "Solo avisar si se cancela").
- Secret de firma: Generado automáticamente para verificar la autenticidad.
- Historial de entregas (Logs): Ver si el webhook fue enviado y la respuesta del servidor destino.
- Botón "Reenviar" (Retry) en caso de fallo.

Entrega Robusta:

- Retries automáticos con Backoff Exponencial (si falla la primera, intenta a los 5 min, luego 30 min, luego 4 horas).
- Timeout de 30 segundos.

REQUISITOS TÉCNICOS:

Lógica de Webhooks:

```
t
typescript
// lib/webhooks.ts
import crypto from 'crypto';
import axios from 'axios';

export async function triggerWebhook(event: string, payload:
any, webhookId: string) {
  const webhook = await prisma.webhook.findUnique({ where: {
    id: webhookId } });
  if (!webhook) return;

  // Verificar si el evento está suscrito
  if (!webhook.events.includes(event)) return;

  const body = JSON.stringify({
    id: crypto.randomUUID(),
    event,
    timestamp: new Date().toISOString(),
    data: payload,
  });

  // Firma HMAC
  const signature = crypto.createHmac('sha256',
webhook.secret).update(body).digest('hex');

  try {
    await axios.post(webhook.url, body, {
      headers: {
        'Content-Type': 'application/json',
        'X-Webhook-Signature': signature,
        'User-Agent': 'TuPlataforma-Webhook/1.0',
      },
      timeout: 30000,
    });

    // Registrar éxito
    await logWebhookDelivery(webhookId, 'success');
  } catch (error) {
    // Registrar error y programar reintento
    await logWebhookDelivery(webhookId, 'failed',
error.message);
    // Aquí llamarías a una cola (Queue) para reintentar
```

```
}  
}
```

Endpoint para Recibir Webhooks (ej. de Zapier a nosotros):

typescript

```
// app/api/webhooks/incoming/route.ts  
import { NextRequest, NextResponse } from 'next/server';  
import crypto from 'crypto';  
  
export async function POST(req: NextRequest) {  
  // Ejemplo: Recibir evento desde un POS externo (Vendty)  
  const signature = req.headers.get('x-pos-signature');  
  const body = await req.text();  
  
  // Verificar firma del POS  
  const isValid = verifySignature(body, signature,  
    process.env.POS_SECRET);  
  
  if (!isValid) {  
    return NextResponse.json({ error: 'Firma inválida' }, {  
      status: 401 });  
  }  
  
  const data = JSON.parse(body);  
  
  // Procesar evento (ej. actualizar estado de mesa si el POS  
  dice que pagaron)  
  if (data.event === 'bill_paid') {  
    await handleBillPaid(data.tableNumber);  
  }  
  
  return NextResponse.json({ received: true }, { status: 200  
});  
}
```

SECCIÓN 3: INTEGRACIONES NATIVAS

REQUISITOS FUNCIONALES:

Conectores "Plug & Play" para servicios populares.

1. Google Reserve (Google Maps):

- Sincronizar disponibilidad y reservas directamente con Google Maps.
- El usuario ve "Reservar" en Google Maps y cae en nuestro sistema (o se sincroniza).
- Requiere validación de Google (Google Business Profile API).

2. Zapier / Make:

- Crear un "Zap" que dispare acciones.
- Trigger: "Nueva Reserva".
- Action: "Crear Cliente en Mailchimp", "Enviar Mensaje a Slack", "Agregar a Google Sheets".

3. POS (Puntos de Venta):

- Vendty / SoftRestaurant: Integración para enviar reservas al POS y que aparezcan como órdenes anticipadas.
- Mapeo de campos (Nombre cliente -> Orden POS).

4. Slack / Discord:

- Notificar al canal del equipo cuando hay una reserva VIP o un evento grande.

REQUISITOS TÉCNICOS:

Integración con Slack (Ejemplo):

```
typescript
// services/integrations/slack.ts
export async function sendSlackNotification(webhookUrl:
string, message: string) {
  await axios.post(webhookUrl, {
    text: message,
    blocks: [
      {
        type: 'section',
        text: {
          type: 'mrkdwn',
          text: ` *Nueva Reserva VIP*\n${message}`
        }
      }
    ]
  });
}
```

SECCIÓN 4: DOCUMENTACIÓN Y SDK

REQUISITOS FUNCIONALES:

Experiencia para desarrolladores (DevEx).

1. Swagger / OpenAPI UI:

- Página en `tudominio.com/api/docs`.
- Documentación interactiva: Puedes probar los endpoints desde el navegador.

- Autenticación directa en el UI (input para pegar el API Key).
 - Esquemas de JSON claros.
2. SDKs (Software Development Kits):
- Generar librerías para facilitar la integración.
 - Node.js SDK: `npm install @tu-plataforma/sdk`
 - Python SDK: `pip install tu-plataforma`
 - Ejemplos de uso en el README.
3. Sandbox / Playground:
- Permitir a los desarrolladores probar sin afectar datos reales.

REQUISITOS TÉCNICOS:

Swagger con next-swagger-doc:

```
typescript
// app/api/docs/page.tsx
import { getApiDocs } from '@lib/swagger';
import ReactSwagger from '@components/react-swagger';

export default async function IndexPage() {
  const spec = await getApiDocs();
  return <ReactSwagger spec={spec} />;
}
```

Ejemplo de Node.js SDK:

```
typescript
// sdk/src/client.ts
export class Client {
  private apiKey: string;
  private baseUrl: string;

  constructor(apiKey: string, baseUrl =
'https://api.tu-plataforma.com/v1') {
    this.apiKey = apiKey;
    this.baseUrl = baseUrl;
  }

  async createBooking(bookingData: BookingPayload) {
    const response = await fetch(`${this.baseUrl}/bookings`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${this.apiKey}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(bookingData),
    });
    return response.json();
  }
}
```

```
}  
}
```

SECCIÓN 5: DASHBOARD DE INTEGRACIONES (UI)

REQUISITOS FUNCIONALES:

Interfaz para que el restaurante gestione sus conexiones.

Pestañas:

1. API Keys:

- Crear/Nombrar claves (ej. "Clave para el desarrollador de la web").
- Copiar clave.
- Revocar claves.

2. Webhooks:

- Lista de URLs suscritas.
- Estado (Activo/Error).
- Log de eventos recientes (éxito/fallo).
- Botón "Agregar Webhook".

3. Integraciones:

- Grid de apps (Google, Slack, Zapier, POS).
- Estado de conexión (Conectado/Desconectado).
- Botón "Configurar".

REQUISITOS TÉCNICOS:

Componentes UI:

- `ApiKeyManager.tsx`: Tabla de claves con modal de creación.
- `WebhookLogs.tsx`: Tabla de logs con filtros por estado y fecha.
- `IntegrationCard.tsx`: Card visual con logo del servicio y toggle de conexión.

RESUMEN DE ENTREGABLES ESPERADOS

Después de implementar este módulo, debes tener:

✓ API Pública REST:

- Endpoints seguros para Reservas, Mesas y Clientes.
- Autenticación por API Key y Rate Limiting.
- Validación estricta con Zod.

✓ Sistema de Webhooks:

- Creación y gestión de endpoints externos.
- Firmas de seguridad (HMAC).
- Historial de entregas y reintentos automáticos.

✓ Integraciones:

- Google Reserve (sincronización básica).
- Zapier (Triggers configurables).
- Slack (Notificaciones).
- Preparación para POS (APIs de entrada).

✓ Documentación:

- Swagger UI interactivo y funcional.
- SDK básico en Node.js.
- Guías de inicio rápido.

✓ Dashboard de Desarrollador:

- Gestión de claves y webhooks.
- Monitoreo de actividad.
- Logs de errores.

INSTRUCCIONES PARA ANTIGRAVITY

1. Implementa el Middleware de API (Auth + Rate Limiting).
2. Crea los Endpoints REST (Bookings, Availability).
3. Desarrolla el servicio de Webhooks (Envío, Firmas, Logs en DB).
4. Construye la UI de configuración (Dashboard > Configuración > Integraciones).
5. Instala y configura Swagger para la documentación automática.
6. Implementa la integración con Slack y Zapier como ejemplos iniciales.
7. Genera el SDK básico en Node.js.

Prioridad:

1. API Segura y Rate Limiting (Fundamental).
2. Webhooks (Necesario para notificaciones externas).

3. Documentación Swagger (Para que otros usen la API).
4. Integraciones específicas (Slack, Zapier).

Nota: La seguridad es crítica aquí. Nunca expongas claves secretas en el frontend y asegúrate de que cada request a la API esté validado contra el restaurante correspondiente.

¿Listo para finalizar el desarrollo con este módulo? Empieza por el Middleware y la estructura de la API. 🚀